

Local Network Discovery

Overview

Lava supports automatic discovery of proxy servers running on your local network. When you open the Settings menu, the app scans for nearby Lava proxy instances and offers to connect to them automatically.

How It Works

mDNS Service Discovery

The Lava proxy advertises itself on the local network using **mDNS** (multicast DNS) with the service type `_lava._tcp`. The Android app uses the platform's `NsdManager` to discover these services.

Discovery Flow

1. **Proxy Advertises:** When the proxy server starts, it registers an mDNS service announcing its IP address and port.
2. **App Scans:** When you open the Settings menu, the app automatically starts a 5-second network scan.
3. **Manual Refresh:** In the connection settings bottom sheet, tap the **refresh icon** (↻) next to the edit button to manually scan for local endpoints at any time.
4. **Auto-Connect:** If a proxy is found and you are not already using a custom mirror, the app automatically adds it to your endpoints and opens the connection settings dialog.
5. **Manual Selection:** You can always switch endpoints manually in the connection settings.

Running a Local Proxy

Using Docker Compose

```
./start.sh
```

This script:

1. Builds the proxy fat JAR
2. Builds a Docker image
3. Starts the container with port `8080` exposed
4. Waits for the service to be healthy

Manual Start

```
./gradlew :proxy:buildFatJar  
java -jar proxy/build/libs/app.jar
```

The proxy listens on `0.0.0.0:8080` and advertises itself via mDNS.

Requirements

- Both devices must be on the **same local network** (same Wi-Fi or LAN).
- Android 5.0+ (API 21+) for the app.
- The proxy host must allow multicast traffic (most home routers do by default).

Troubleshooting

Issue	Solution
Proxy not discovered	Tap the refresh icon in connection settings to trigger a manual scan. Ensure both devices are on the same network. Check firewall rules for mDNS (port 5353/UDP).
Discovery is slow	mDNS can take a few seconds. The scan timeout is 5 seconds.
Auto-connect didn't happen	If you already have a custom mirror configured, the app will not auto-switch. Manually select the discovered endpoint in connection settings.
mDNS blocked on corporate networks	Some corporate networks block multicast. Use manual endpoint entry instead.

On-device API instances (platform=android)

In addition to a host-run `lava-api-go` server, a phone or tablet running the **Lava API Android app** can itself become a LAN-reachable API (see [ON_DEVICE_API.md](#)). It advertises on the same service types the client already watches, distinguished only by a TXT key:

Source	Service type	TXT records
Host server (release)	<code>_lava-api._tcp</code>	<code>engine=go, platform=server</code> (or absent)
Host server (dev)	<code>_lava-api-dev._tcp</code>	<code>engine=go-dev</code>
On-device app (release)	<code>_lava-api._tcp</code>	<code>engine=go, platform=android, storage=sqlite, version=<n></code>
On-device app (dev)	<code>_lava-api-dev._tcp</code>	<code>engine=go-dev, platform=android, storage=sqlite</code>

The client discovery (`LocalNetworkDiscoveryServiceImpl`) already resolves these to `Endpoint.GoApi(host, port)` via the authoritative TXT engine attribute (falling back to the service type). Instances without a `platform` key render exactly as today, so this is fully backward compatible. The client labelling that calls out "an Android device on this network" using the `platform=android` key is **PENDING sub-project 2**.

PENDING (Phase D): the on-device *advertiser* (the embed registering itself via `NsdManager`) is not yet in the tree. The client-side discovery described above is in-tree and unchanged.

Authentication when connecting to a discovered API

Discovery only finds an instance; **connecting requires the Lava-Auth key**. Every `lava-api-go` instance — host-run or on-device — enforces a Lava-Auth gate (HMAC-SHA256 over a base64 UUID credential in the Lava-Auth header, with a per-IP backoff ladder). `/health` and `/ready` answer without a credential so discovery probes work, but any data endpoint returns 401 without the correct key. For an on-device API, the key is generated (or supplied) by the API app and shown for pairing — see the [user guide](#).